Google Ads API Integration Brief

Document Classification: Internal Engineering Manual
Target Audience: Core Backend & DevOps Teams

1. System Architecture & OAuth 2.0 User Flow

To build a global SaaS platform where users manage ads from our frontend dashboard, we cannot hardcode static developer credentials. We must use a **Multi-Tenant OAuth 2.0 Flow** to safely act on behalf of our users.

The Connection Sequence:
1. **User Action:** The client clicks 'Connect Google Ads Account' on our frontend dashboard.
2. **Authorization Request:** Our backend redirects the user to Google's OAuth secure server with our client_id, scopes (https://www.googleapis.com/auth/adwords), and access_type=offline.
3. **User Grant:** The user logs in and approves our platform's requested permissions.
4. **Token Exchange:** Google sends a short-lived authorization_code to our backend redirect URI. Our backend immediately sends a POST request to swap this code for an Access Token (short-lived) and a Refresh Token (long-lived).
5. **Database Storage:** We encrypt and save the Refresh Token alongside the user's unique 10-digit Google Ads Customer ID in our centralized database.

2. The 5-Step Campaign Mutate Stack

The Google Ads API is highly relational and strictly atomic. You cannot create an ad creative without linking it to an explicit hierarchy. Your backend code must execute these mutations sequentially, capturing the generated resource IDs at each step to build the relational tree structure:
• **Step 1: CampaignBudgetService** → Generates and returns a unique Budget ID.
• **Step 2: CampaignService** → Requires Budget ID; Sets channel type and bidding strategy.
• **Step 3: AdGroupService** → Container inside the campaign instance to organize groups.
• **Step 4: AdGroupCriterionService** → Injects targeted keyword configurations.
• **Step 5: AdGroupAdService** → Uploads final asset arrays (headlines, copy descriptions).

```
from google.ads.googleads.client import GoogleAdsClient
from google.ads.googleads.errors import GoogleAdsException

def create_google_ads_campaign(client, customer_id):
    try:
        # STEP 1: Define and Create the Campaign Budget
        budget_service = client.get_service("CampaignBudgetService")
        budget_operation = client.get_type("CampaignBudgetOperation")
        budget = budget_operation.create
        budget.name = "SaaS Automated Daily Budget"
        # Google Ads API uses micros (Units multiplied by 1,000,000)
        budget.amount_micros = 500000000
        budget.delivery_method = client.enums.BudgetDeliveryMethodEnum.STANDARD

        budget_response = budget_service.mutate_campaign_budgets(customer_id,
[budget_operation])
        budget_resource_name = budget_response.results[0].resource_name

        # STEP 2: Instantiate the Campaign Shell
        campaign_service = client.get_service("CampaignService")
```

```
        campaign_operation = client.get_type("CampaignOperation")
        campaign = campaign_operation.create
        campaign.name = "Global Scale Performance Max Campaign"
        campaign.advertising_channel_type =
client.enums.AdvertisingChannelTypeEnum.PERFORMANCE_MAX
        campaign.status = client.enums.CampaignStatusEnum.PAUSED
        campaign.campaign_budget = budget_resource_name

        # IMPLEMENT SMART BIDDING STRATEGY (From Dropdown Choice)
        campaign.maximize_conversions = client.get_type("MaximizeConversions")

        campaign_response = campaign_service.mutate_campaigns(customer_id,
[campaign_operation])
        return campaign_response.results[0].resource_name
    except GoogleAdsException as ex:
        print(f"API Exception: {ex.status.message}")
```

3. Data Ingestion: Analytics via GAQL Engine

To populate our custom user interface charts, we submit high-performance, SQL-like syntax streams using the Google Ads Query Language (GAQL) structure.

```
def fetch_campaign_performance(client, customer_id):
    ga_service = client.get_service("GoogleAdsService")
    query = """
        SELECT campaign.id, campaign.name, metrics.impressions, metrics.clicks,
metrics.cost_micros
        FROM campaign WHERE segments.date DURING LAST_30_DAYS
    """
    stream = ga_service.search_stream(customer_id=customer_id, query=query)
    for batch in stream:
        for row in batch.results:
            cost_actual = row.metrics.cost_micros / 1000000
            print(f"{row.campaign.name}: {row.metrics.clicks} Clicks | Cost: {cost_actual}")
```

4. Production Mitigations & Limits

Challenge	Root Cause	Solution Architecture
Rate Limiting	Exceeding operation calls on basic access tiers.	Implement backend job task queuing (Redis/Celery) to pace out automated mutates.
Policy Rejection	Users injecting blacklisted copywriting inputs.	Call validate_only=true execution parameters before forcing final production uploads.
Token Expiration	Access tokens automatically timeout every 3600 seconds.	Build secure server middleware hooks to auto-refresh access windows invisibly via refresh tokens.